

Number: P3818R2

Date: 2026-05-12

Audience: [Library Evolution](#) and [Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

-
- [Revision history](#)
 - [Motivation](#)
 - [Past discussions](#)
 - [Potentially-constant initialization](#)
 - [Explainer table](#)
 - [constexpr variables are not a problem](#)
 - [Immediate \(constexpr\) escalation](#)
 - [Proposed solution](#)
 - [Possible alternatives](#)
 - [Much larger but probably breaking solution](#)
 - [Duplicate functions under different name specially for constant evaluation](#)
 - [Runtime and constexpr exception shouldn't have different semantic](#)
 - [One function name is needed](#)
 - [Allow current_exception only in exception handler](#)
 - [Not change std::uncaught_exception now](#)
 - [Constant evaluation shouldn't diverge](#)
 - [Implementation experience](#)
 - [GCC](#)
 - [Wording](#)
 - [Feature test macros](#)

P3818R2: constexpr exception library which is unsurprising in potentially constant initialization

Previous version of this paper was originally considered as a bugfix for C++26, but LEWG decided it's more prudent thing to do to just remove constexpr modifiers on `std::uncaught_exceptions()` and `std::current_exception()`, part of the final change in the removal was also making `std::nested_exception` not constexpr.

This paper proposes solution to surprising silent code breakage introduced by [P3068 "constexpr exceptions"](#) interacting with [potentially-constant initialization \[expr.const\]](#). This was [found](#) by Lénárd Szolnoki and discussed at library and core wording groups reflectors. In C++26 these functions are non-constexpr, which makes impossible to store an exception and use it later.

This is also prerequisite for proper work of constexpr coroutines, like exception being thrown out of `std::generator`.

Revision history

- [R1](#) → R2: targetting C++29 instead, change of wording to add removed constexpr.
- [R0](#) → [R1](#): added [section discussing changes proposed in P3820](#), added [comparison table](#) to explaining subtle differences between various types of variables and their initialization, added [comparison table](#) between P3818 and P3820 proposals.

Motivation

To make constexpr exception support complete, and allow all functionality withing constant evaluation. This paper was seen in [previous revision](#) by LEWG. It didn't get consensus and most of the group preferred the conservative approach of removing constexpr from `std::uncaught_exception()` and `std::current_exception()`. The group asked me to revisit this again for C++29.

Once we get constexpr coroutines and `std::generator` following code in constant evaluation (because it works with `std::meta::info`) must work:

```
// before: std::generator can't throw, no coroutine can throw
// after: this works intuitively even with exceptions
constexpr std::generator<std::meta::info> analyze_members(std::meta::info);

constexpr auto analyze_type(std::meta::info type) {
    try {
        for (auto v: analyze_members(type)) { // generator's iterator can throw when de
referencing
            // ...
        }
    } catch (const std::meta::exception & e) {
        // a recovery code
    }
}
```

Past discussions

LEWG on 2025/09/09 telecon

***POLL:** Considering that we are already in the NB comment phase for C++26, adopt the solution laid out in P3818R1 (special handling for potentially constant initialization for both `uncaught_exceptions` and `current_exception`) as a solution to the problem for C++26.*

SF	F	N	A	SA
5	6	2	4	3

Outcome: No consensus.

***POLL:** Considering that we are already in the NB comment phase for C++26, remove `constexpr` from `uncaught_exceptions` and make `current_exception` conditionally constant evaluated solution to the problem for C++26 (like P3820, but without the `consteval` functions, requires a new paper)*

SF	F	N	A	SA
2	2	2	9	5

Outcome: Consensus against.

***POLL:** Remove the `constexpr` from `uncaught_exceptions` and `current_exception` for C++26*

SF	F	N	A	SA
7	9	3	2	0

Outcome: Consensus in favor.

During the discussion in LEWG it was discussed [we want to revisit](#) this for C++29.

Potentially-constant initialization

C++ has many corner cases and this one is one of them. A constant variable (marked `const`) which is integral or enumeration type is upgraded to `constexpr` variable silently if its initialization succeed in constant evaluation. This is usually unobservable, because you can't reference anything around to succeed.

```
auto function_returning_empty_array() {
    const int n = calculate_size_needed(); // this needs to be a constant evaluated =
    > constexpr
    return std::array<int, n>{}; // so we can change type based on `n`
}
```

This is a problem for `constexpr` exceptions, which needs `constexpr` marked functions in order for them work inside constant evaluation. But once marked `constexpr` a function can be evaluated there, which is a problem for two functions added by [P3068](#) (`std::uncaught_exceptions` and `std::current_exceptions`) as these don't have dependency on any local variable which would disallow constant evaluation. These have only an implicit dependency on local context which allows them to be successfully evaluated in `const` variable potentially-constant initialization.

These two functions were stripped of `constexpr` modifier before releasing C++26 out of concerns of late changing behaviour with this paper.

This potentially-constant initialization starts as any constant evaluation a new context, in which there are no unrolling or current exception, so these function will return constant which is something else user wants.

```
try {
    // some exception throwing code
} catch (const std::exception & exc) {
    const bool has_exception = (std::current_exception() != nullptr); // no current
exception in a catch handler?!
    static_assert(has_exception == false); // success here
}
```

Variable `has_exception` is silently upgraded to a `constexpr` variable. And records different context than a user expects. Another even more scary example:

```
struct transaction {
    // ...
    void cancel() { /* revert changes */ }

    ~transaction() {
        const bool unrolling = std::uncaught_exceptions() > 0;

        if (unrolling) { // this will never be evaluated
            log("exception was thrown in a transaction => cancel().");
            cancel();
        }
    }
}
```

Note: I know this example should contain call in `std::uncaught_exceptions()` so we can actually know if there is a new exception, but in order to simplify it I did what I did.

In previous example `std::uncaught_exceptions() > 0` is constant evaluated in a vacuum, even sooner than the destructor `transaction::~~transaction()` is finished parsing. And because in that specific constant evaluation, there is no uncaught exception, it will return 0, *obviously*. The whole unrolling becomes `constexpr`, and your transactions will **never** cancels. This is really scary.

Explainer table

potentially-constant initialization (sometimes constant evaluated without visible clue)			
	C++23	C++26 (status quo)	P3818 (this paper for C++29)
<code>const int n = uncaught_exception()</code> (local variable)	number of exceptions (during runtime evaluation)	number of exceptions* (ill-formed in constant evaluation)	number of exceptions (during runtime <u>and</u> <u>constant</u> evaluation)
<code>static const int n = uncaught_exception()</code> (local static variable)	number of exceptions (during <u>first</u> runtime evaluation, static const variables are not allowed in constant evaluated functions)		
<code>const int n = uncaught_exception()</code> (object member variable)	number of exceptions (during evaluation of object's constructor; never was a constant evaluation, unchanged)	not supported (removed from C++26)	number of exceptions (during evaluation of object's constructor; will <u>newly</u> work if the object is used during constant evaluation)
explicit constant evaluation			
	C++23	C++26 (status quo)	P3818 (this paper)
<code>constexpr int n = uncaught_exception()</code> (local constant)	not available (constexpr exception aren't available)	not available (constexpr exceptions are available, questionable utility)	always 0 (constant evaluated during parsing)
<code>static constexpr int n = uncaught_exception()</code> (static constant)			always 0 (constant evaluated during parsing)
<code>static_assert(uncaught_exception() == 0)</code>			always true (constant evaluated during parsing)

<pre>auto obj = some_template<uncaught_exception()> {}</pre>			some_template<0> (constant evaluated during parsing)
<pre>constexpr auto obj = some_function() (using std::uncaught_exception internally)</pre>		ill-formed (functions not marked constexpr)	some_function() will work (constant evaluated during parsing)

The code in table is intentionally visible, I don't expect users to write `constexpr int n = std::uncaught_exceptions()` but I do expect users to write code which is using the function deep inside the code, not visibly.

constexpr variables are not a problem

It can be surprising to some users a local `constexpr` variables are not observing local evaluated context. But it's long established all `constexpr` variables are starting completely new evaluation without any evaluation context at site of declaration (it can use template variables, other `constexpr` variables, but not any local variable).

We need to keep `constexpr` marked functions in order to have the `constexpr` exception functionality fully working during constant evaluation (storing exceptions temporarily). Failing to do so would make a somehow arbitrary functionality of language again impossible to use and for users to go around, which I strongly prefer to avoid so.

In order to do we must make the potentially-constant initialization evaluation fail when it reaches these two function in question. It's a small surgical and mostly inobservable change which saves us from the silent code change when upgrading to 26. But also it's much better than just removing `constexpr`.

Immediate (consteval) escalation

Immediate escalation happen only to `template-d` `constexpr` function and `lamdas`, when they has `consteval` only value of `std::meta::info` or taking a function pointer of `consteval` function.

Such functions are then implicitly `consteval` with all its implications. Like when you call a `consteval` function from a non-`consteval` function (these needs to have a `codegen`, but `consteval` has no `codegen`) you will get an error. Unless the function is indendent on its context (all its arguments can be constant evaluated too, or is without any argument), then the function call site is replaced with its result. Look at the following code:

```
constexpr int CONSTEXPR_number_of_exceptions() {
    return std::uncaught_exceptions();
}

constexpr int CONSTEVAL_number_of_exceptions() {
    return std::uncaught_exceptions();
}

template <typename T = void> constexpr int ESCALATED_number_of_exceptions() {
    auto escalate_me = ^^T; // to force escalation
    return std::uncaught_exceptions();
}

struct guard {
    constexpr ~guard() {
    {
        int mutable_v          = CONSTEXPR_number_of_exceptions();
        const int immutable_v   = CONSTEXPR_number_of_exceptions();
        constexpr int constant_v = CONSTEXPR_number_of_exceptions();
        std::println("constexpr: {}", mutable_v, immutable_v, constant_v);
    }
    {
        int mutable_v          = CONSTEVAL_number_of_exceptions();
        const int immutable_v   = CONSTEVAL_number_of_exceptions();
        constexpr int constant_v = CONSTEVAL_number_of_exceptions();
        std::println("constexpr: {}", mutable_v, immutable_v, constant_v);
    }
    {
        int mutable_v          = ESCALATED_number_of_exceptions();
        const int immutable_v   = ESCALATED_number_of_exceptions();
        constexpr int constant_v = ESCALATED_number_of_exceptions();
        std::println("escalated: {}", mutable_v, immutable_v, constant_v);
    }
    }
};

int test() {
    const auto g = guard{};
    throw 42;
}
```

This code prints following result:

In C++26:

```
constexpr: 1, 0, ?
constexpr: ?, ?, ?
escalated: ?, ?, ?
```

After this paper:

```
constexpr: 1, 1, 0
constexpr: 0, 0, 0
escalated: 0, 0, 0
```

Notice the questionmarks **?** in "before" which marks things which are currently impossible to code. Also notice the **1** in second column for constexpr variant, it takes its runtime / current constant context (not starting a new one).

This is a result of language rule with escalation. And there is no way the constant evaluated escalated function could observe its runtime context, after all, it must not be runtime evaluated, it become `constexpr`.

Proposed solution

Keep constexpr on both methods (`std::uncaught_exceptions()` and `std::current_exceptions()`) and **disallow them** to be constant evaluated explicitly only in **potential-constant initialization** [`expr.const`].

This is a minimal and implemented solution which doesn't limit functionality, but removes the break.

Possible alternatives

Much larger but probably breaking solution

We could deprecate and later remove `potentially-constant` from language. This would make C++ much less surprising, but it will be probably a significant breaking change, although not really hard to fix (just make your `const` variables which suddenly failed to compile `constexpr` and you are good to go.)

Because of the large impact, this is not proposed.

Duplicate functions under different name specially for constant evaluation

This is what P3820R0 proposes and [LEWG rejected it](#). Removing `constexpr` from `uncaught_exceptions` (not from `current_exception` as that one is not touched by the paper) and introduce a same function under similar name. I think that's a bad solution, because these functions are doing same thing, problem is interaction with language rules. Also I really don't think we should introduce same functionality which then would need to be `constexpr`-ed on every place where we want to use code for both runtime and constant evaluation. It will lead to code like this:


```
constexpr foo::~~foo() {
    if consteval {
        /* const */ int num_of_exceptions = std::consteval_uncaught_exceptions();
        if (num_of_exceptions != num_of_exceptions_before) {
            // do something now we know there is an exception unrolling
        }
    } else {
        const int num_of_exceptions = std::uncaught_exceptions();
        if (num_of_exceptions != num_of_exceptions_before) {
            // do something now we know there is an exception unrolling
        }
    }
}
```

Previous example shows multiple problems, even if we do introduce new function, we can still can't use it in `const int` variable initialization, because it would create new constant evaluation and fold into constant. We must avoid `const int` there completely and we must teach it then.

I fully expect users to write something like following function:

```
constexpr int my_uncaught_exceptions() {
    if consteval {
        // for some reason committee gave us this function,
        // but I need this to initialize variable, so I can't use
        // if consteval there
        return std::consteval_uncaught_exceptions();
    } else {
        return std::uncaught_exceptions();
    }
}
```

And later someone will use this function in `const int n = my_uncaught_exceptions` initialization and gets burned anyway. Therefore I argue against creating new variable, instead I propose taking proposal of this paper, which will make sure there is no breaking change.

Runtime and constexpr exception shouldn't have different semantic

There is no difference semantic in how runtime and constexpr exceptions behave. Hence providing different namely function to do the same just in different context will lead to confusion and previously describe problems. Difference is on the language level between local variables (`int n = ...`) and local constants (`constexpr int n = ...`) and when these are initialized and evaluated. With future support for compile time debug printing from [P2758](#) users will be able observe when is what code evaluated, and will understand that code evaluated in their compiler can't logically know about number of uncaught exceptions during runtime evaluation.

One function name is needed

If we split functions to two, we are forcing users to write their own "one common function". If they want to continue support both types of their exceptions in pattern where number of unrolling exceptions is stored in a member variable for later comparison in destructor. This shows it's same functionality and it would burden users.

Allow `current_exception` only in exception handler

This would need to be also transitive and it will introduces non-observable execution state, which can make some constant evaluation to fail irrecoverably. This doesn't solve problem of the `std::uncaught_exceptions()` at all.

I think it's essential to keep functionality "give me an exception if there is any" available, and not make it "give me an exception or fail compilation", ability to recover must be provided.

This wouldn't solve the original problem:

```
template <typename T = int> constexpr bool bar() {
    std::meta::info var = ^^T;
    // same applies if `bar` is simple consteval function
    return std::current_exception() != nullptr;
}

constexpr bool foo() { // this needs to have codegen
    try {
        throw 42;
    } catch (...) {
        bool v = bar(); // notice this call to escalated / consteval function
        return v;
    }
}

static_assert(foo() == true); // constant evaluation where `bar()` is evaluated as
// part of static_assert's evaluation, therefore in constant evaluator

int main() {
    const bool v = foo(); // true
    bool y = foo(); // failure to compile, because `current_exception()`
    // is called outside exception handler
}
```

Not change `std::uncaught_exception` now

If you consider `uncaught_exceptions()` so broken it needs to fix and you want it to be fixed, we can skip making it `constexpr` now, but still make `current_exception()` `constexpr`.

Constant evaluation shouldn't diverge

There is a trend of running tests in the constant evaluator (I also prototyped constexpr code coverage measurement), where you are supposed to detect all UBs. Making this testing not mirror the runtime behaviour would break this model (yes, I don't like `if constexpr` too, and I don't think users should be forced to use it at all).

Implementation experience

The proposed solution was implemented in [my clang prototype of constexpr exception](#) for `std::uncaught_exceptions()`, and you can experiment with [it at the compiler explorer](#).

The change itself was add to clang know in its evaluation state the evaluation is potentially-constant. And then the builtins implementing the exception handling function to detect it and fail to evaluate in that case.

GCC

I have talked with Jakub and he came to same solution as proposed here.

Wording

Change is add constexpr modifier and *Constant when* to `std::uncaught_exceptions()` and `std::current_exception()`. Also it returns back constexpr of nested_exception which was removed by [P3842R2](#) "A conservative fix for constexpr uncaught_exceptions() and current_exception()".

17.9 Exception handling

[support.exception]

17.9.2 Header <exception> synopsis

[exception.syn]

```
// all freestanding
namespace std {
    class exception;
    class bad_exception;
    class nested_exception;

    using terminate_handler = void (*)();
    terminate_handler get_terminate() noexcept;
    terminate_handler set_terminate(terminate_handler f) noexcept;
    [[noreturn]] void terminate() noexcept;
```

```
constexpr int uncaught_exceptions() noexcept;

using exception_ptr = unspecified;

constexpr exception_ptr current_exception() noexcept; // exposition only
y
constexpr exception_ptr current_exception() noexcept;
[[noreturn]] constexpr void rethrow_exception(exception_ptr p);
template<class E> constexpr exception_ptr make_exception_ptr(E e) noexcept;
template<class E>
    constexpr optional<const E&> exception_ptr_cast(const exception_ptr& p) noexcept;
    template<class E> void exception_ptr_cast(const exception_ptr&&) = delete;

template<class T> [[noreturn]] constexpr void throw_with_nested(T&& t);
template<class E> constexpr void rethrow_if_nested(const E& e);
}
```

17.9.6 uncaught_exceptions

[uncaught.exceptions]

```
constexpr int uncaught_exceptions() noexcept;
```

Constant When: not evaluated within potentially-constant [expr.const] initialization.

Returns: The number of uncaught exceptions ([except.throw]) in the current thread.

Remarks: When `uncaught_exceptions() > 0`, throwing an exception can result in a call of the function `std::terminate`.

17.9.7 Exception propagation

[propagation]

```
using exception_ptr = unspecified;
```

The type `exception_ptr` can be used to refer to an exception object.

`exception_ptr` meets the requirements of *Cpp17NullablePointer* (Table 36).

Two non-null values of type `exception_ptr` are equivalent and compare equal if and only if they refer to the same exception.

The default constructor of `exception_ptr` produces the null value of the type.

5

exception_ptr shall not be implicitly convertible to any arithmetic, enumeration, or pointer type.

6 [Note 1: An implementation can use a reference-counted smart pointer as exception_ptr. — end note]

7 For purposes of determining the presence of a data race, operations on exception_ptr objects shall access and modify only the exception_ptr objects themselves and not the exceptions they refer to. Use of rethrow_exception or exception_ptr_cast on exception_ptr objects that refer to the same exception object shall not introduce a data race.

[Note 2: If rethrow_exception rethrows the same exception object (rather than a copy), concurrent access to that rethrown exception object can introduce a data race. Changes in the number of exception_ptr objects that refer to a particular exception do not introduce a data race. — end note]

8 All member functions are marked constexpr.

```
constexpr exception_ptr current_exception() noexcept;  
constexpr exception_ptr current_exception() noexcept;
```

? **Constant When:** not evaluated within potentially-constant [expr.const] initialization, unless called from make_exception_ptr.

9 **Returns:** An exception_ptr object that refers to the **currently handled exception** or a copy of the currently handled exception, or a null exception_ptr object if no exception is being handled. The referenced object shall remain valid at least as long as there is an exception_ptr object that refers to it. If the function needs to allocate memory and the attempt fails, it returns an exception_ptr object that refers to an instance of bad_alloc. It is unspecified whether the return values of two successive calls to current_exception refer to the same exception object.

[Note 3: That is, it is unspecified whether current_exception creates a new copy each time it is called. — end note]

If the attempt to copy the current exception object throws an exception, the function returns an exception_ptr object that refers to the thrown exception or, if this is not possible, to an instance of bad_exception.

[Note 4: The copy constructor of the thrown exception can also fail, so the implementation can substitute a bad_exception object to avoid infinite recursion. — end note]

```
[[noreturn]] constexpr void rethrow_exception(exception_ptr p);
```

10 **Preconditions:** p is not a null pointer.

11

Effects: Let u be the exception object to which p refers, or a copy of that exception object. It is unspecified whether a copy is made, and memory for the copy is allocated in an unspecified way.

- (11.1) — If allocating memory to form u fails, throws an instance of `bad_alloc`;
- (11.2) — otherwise, if copying the exception to which p refers to form u throws an exception, throws that exception;
- (11.3) — otherwise, throws u .

```
template<class E> constexpr exception_ptr make_exception_ptr(E e) noexcept;
```

12 *Effects:* Creates an `exception_ptr` object that refers to a copy of e , as if:

```
try {  
    throw e;  
} catch(...) {  
    return current_exceptioncurrent_exception();  
}
```

```
template<class E>  
constexpr optional<const E&> exception_ptr_cast(const exception_ptr&  
p) noexcept;
```

13 *Mandates:* E is a cv-unqualified complete object type. E is not an array type. E is not a pointer or pointer-to-member type.

[Note 5: When E is a pointer or pointer-to-member type, a handler of type `const E&` can match without binding to the exception object itself. — end note]

14 *Returns:* An `optional` containing a reference to the exception object referred to by p , if p is not null and a handler of type `const E&` would be a match (`[except.handle]`) for that exception object. Otherwise, `nullopt`.

17.9.8 nested_exception [except.nested]

```
namespace std {  
    class nested_exception {  
    public:  
        constexpr nested_exception() noexcept;  
        constexpr nested_exception(const nested_exception&) noexcept = default;  
        constexpr nested_exception& operator=(const nested_exception&) noexcept = default;  
        virtual constexpr ~nested_exception() = default;  
  
        // access functions  
        [[noreturn]] constexpr void rethrow_nested() const;
```

```
constexpr exception_ptr nested_ptr() const noexcept;
};

template<class T> [[noreturn]] constexpr void throw_with_nested(T&&
t);
template<class E> constexpr void rethrow_if_nested(const E& e);
}
```

- 1 The class `nested_exception` is designed for use as a mixin through multiple inheritance. It captures the currently handled exception and stores it for later use.
- 2 [Note 1: `nested_exception` has a virtual destructor to make it a polymorphic class. Its presence can be tested for with `dynamic_cast`. — end note]

```
constexpr nested_exception() noexcept;
```

- 3 *Effects:* The constructor calls `current_exception()` and stores the returned value.

```
[[noreturn]] constexpr void rethrow_nested() const;
```

- 4 *Effects:* If `nested_ptr()` returns a null pointer, the function calls the function `std::terminate`. Otherwise, it throws the stored exception captured by `*this`.

```
constexpr exception_ptr nested_ptr() const noexcept;
```

- 5 *Returns:* The stored exception captured by this `nested_exception` object.

```
template<class T> [[noreturn]] constexpr void throw_with_nested(T&& t);
```

- 6 Let `U` be `decay_t<T>`.

- 7 *Preconditions:* `U` meets the *Cpp17CopyConstructible* requirements.

- 8 *Throws:* If `is_class_v<U> && !is_final_v<U> && !is_base_of_v<nested_exception, U>` is `true`, an exception of unspecified type that is publicly derived from both `U` and `nested_exception` and constructed from `std::forward<T>(t)`, otherwise `std::forward<T>(t)`.

```
template<class E> constexpr void rethrow_if_nested(const E& e);
```

- 9 *Effects:* If `E` is not a polymorphic class type, or if `nested_exception` is an inaccessible or ambiguous base class of `E`, there is no effect. Otherwise, performs:

```
if (auto p = dynamic_cast<const nested_exception*>(addressof
(e)))
    p->rethrow_nested();
```

Feature test macros

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_constexpr_exceptions 20250226LL // also in <exception>, <stdexcept>, <expected>, <optional>, <variant>, and <format>
```